Report for Laboratory 7: Motor Velocity Control

Julian Soh

Department of Mechanical Engineering, Saint Martin's University
ME/EE 477—Embedded Real-Time Computing for Mechanical Engineers

July 7, 2026

Abstract. This lab implemented a program that is a closed-loop control system whereby a motor that was being driven was measured for (actual) velocity. This velocity was continuously compared to the input (desired) velocity. The difference between the two velocities was used to compute a control signal that was sent to the amplifier and the plant driving the motor to adjust its speed, hence the closed-looped nature of the system.

1 Introduction

This laboratory exercise focused on building a motor-speed control application as an integrated closed-loop system. The controller continuously compared the commanded velocity with the measured motor velocity and updated the drive signal in real time to reduce the tracking error. The target closed-loop architecture used in this report was shown in Figure 1.

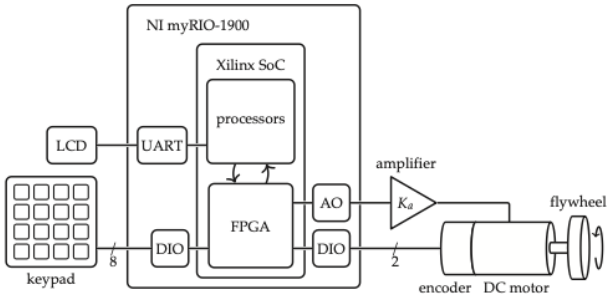

Figure 1: Schematic diagram of the target closed-loop system (Picone et al., 2024).

The objectives of this exercise were to:

1. Integrate the hardware and software components developed in earlier work into a unified closed-loop control system
2. Implement a program structure that supports continuous adjustment of control parameters without stopping the control algorithm
3. Implement a proportional-integral (PI) velocity control algorithm for the DC motor

2 Materials and Methods

2.1 Materials

The materials used in this experiment were:

- Windows 10 computer with myRIO support for C and Eclipse IDE.
- NI myRIO-1900
- Keypad
- Serial display
- PMDC motor
- Analog current amplifier
- DC power supply
- Quadrature encoder
- Flywheel
- Noncontact tachometer
- Oscilloscope (optional)[1]

2.2 Main Function Implementation

The `main()` routine coordinated initialization, user interaction through the table editor, and orderly shutdown of the timer-driven control thread. Its sequence was:

1. Initialize all table-editor variables.
2. Configure, install, and enable the timer IRQ.
3. Register and create the timer thread that serviced the interrupt. For this lab, the thread resource structure included a pointer to the table data so the ISR-side logic could access current parameters.
4. Launch the table editor using six entries:

- `V_R`: rpm {edit}
- `V_J`: rpm {show}
- `VDAout`: mV {show}
- `Kp`: V-s/r {edit}
- `Ki`: V/r {edit}
- `BTI`: ms {edit}

All editable values were initialized to zero, except the BTI setting, which was initialized to 5 ms. After the table editor exited, `main()` signaled the timer thread to stop and then waited until that thread terminated cleanly.

2.3 Interrupt Service Routine (ISR) Thread

At the top of the timer-thread entry function, convenient aliases were created for table entries through the table pointer, for example:

[1] The oscilloscope was connected to AO0 to monitor the power output from the myRIO for troubleshooting purposes and for further demonstration in Problem L7.5.

```
double *Omega_R = &((threadResource->a_table + 0)->
    value);
double *Omega_J = &((threadResource->a_table + 1)->
    value);
// etc.
```

The timer thread then ran in an IRQ-paced main loop and exited only when its ready flag was cleared. Before entering the loop, it:

- Initializes analog I/O and forces motor command voltage to 0 V with `Aio_Write()`.
- Configures the encoder counter interface.

During each interrupt period, the thread performed the following operations:

1. Prepare for the next sampling instant by waiting for IRQ assertion, writing the Timer Write Register, and writing `TRUE` to the Timer Set Time Register.
2. Call `vel()` (from Lab 4) to obtain the current motor velocity.
3. Compute the PI-law coefficients (a_i and b_i) from the current K_P and K_I values using Eq. (2), then update the active biquad-coefficient structure.
4. Compute instantaneous speed error as $\Omega_R - \Omega_J$.
5. Call `cascade()` to evaluate the PI control output from the current error via the difference equation, then saturate the command to the range $[-10.0, +10.0]$ V.
6. Send the control voltage to DAC connector C, channel AOC0, using `Aio_Write()`.
7. Update table show-fields so the displayed values track current controller conditions.
8. Store the current BTI results for later analysis.
9. Acknowledge the interrupt.

2.4 Implementing the PI Controller

The PI controller mapped velocity error $e(t)$ to the control command $u_a(t)$ through proportional and integral gains, K_P and K_I. The closed-loop signal flow used in this experiment was shown in Figure 2, including the controller, amplifier, plant, and encoder feedback path (Picone et al., 2024).

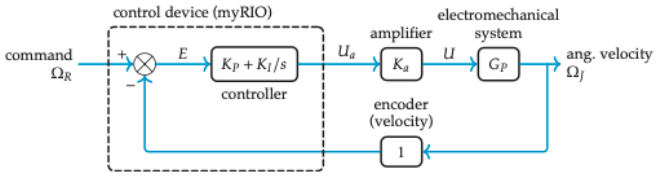

Figure 2: Control block diagram for the Lab 7 PI velocity-control system (Picone et al., 2024).

Starting from the continuous-time controller model,

$$G_C(s) = K_P + \frac{K_I}{s}, \tag{1}$$

and applying the Tustin transformation, the discrete controller could be written as

$$G_C(z) = \frac{U_a(z)}{E(z)} = \frac{b_0 + b_1 z^{-1}}{a_0 + a_1 z^{-1}}, \tag{2}$$

with coefficients

$$a_0 = 1, \quad a_1 = -1, \quad b_0 = K_P + \frac{K_I T}{2}, \quad b_1 = -K_P + \frac{K_I T}{2}, \tag{3}$$

where T was the sampling period. The discrete velocity error used by the controller was

$$E(n) = \Omega_R(n) - \Omega_J(n). \tag{4}$$

3 Laboratory Exploration and Evaluation

3.1 Problem L7.1

Design a PI controller for the T1 target system using the root-locus procedure from Sections 7.6.2–7.6.6, then compare the resulting K_P and K_I values with the base gain set computed for this laboratory.

The base PI gains for the T1a target system were computed from the design relations

$$K_P = \frac{B + 2J\,\mathcal{R}(\psi)}{K_a K_M}, \tag{5}$$

$$K_I = \frac{Z\,(B + 2J\,\mathcal{R}(\psi))}{K_a K_M}. \tag{6}$$

Using the T1a parameters,

$$K_a = 0.06\ \mathrm{A/V}, \quad K_M = 0.0698\ \mathrm{N\,m/A}, \quad J = 7.8 \times 10^{-6}\ \mathrm{N\,m\,s^2/rad}, \tag{7}$$

the combined amplifier-motor gain was

$$K_a K_M = (0.06)(0.0698) = 0.004188. \tag{8}$$

Substituting (7) and (8) into (5) and (6), with the root-locus design selections used for the T1a case, gives

$$K_P = 0.104\ \mathrm{V\,s/rad}, \quad K_I = 2.07\ \mathrm{V/rad}. \tag{9}$$

These values were consistent with the baseline gains reported for Lab 7 and were used as the starting controller settings in the remainder of the experiment. The controller implementation used velocity in rad/s, which was consistent with the units of the computed gains.

The base PI gains were $K_P = 0.104$ V s/rad and $K_I = 2.07$ V/rad.

3.2 Problem L7.2

Design a PI controller for your specific T1 target system following the root locus design technique of Sections 7.6.2 to 7.6.6. Compare the gains K_P and K_I to the base set of gains computed in Lab Problem L7.1.

The PI controller for the T1a target system was designed using the root-locus technique described in Sections 7.6.2–7.6.6. From the uncompensated root locus, a gain of

$$K_1 = 0.062 \tag{10}$$

was selected at the desired pole location

$$\Re(s) = -20. \tag{11}$$

A PI compensator with zero location

$$Z_I = -20 \tag{12}$$

was then introduced. The compensated root locus required an additional gain

$$K_2 = 1.67. \tag{13}$$

The resulting PI controller gains were computed as

$$K_P = K_1 K_2 = (0.062)(1.67) = 0.10354 \text{ V s/rad} \tag{14}$$

and

$$K_I = -Z_I K_P = 20(0.10354) = 2.0708 \text{ V/rad}. \tag{15}$$

These values were compared with the base controller gains from Problem L7.1.

Gain	Base Gains (L7.1)	Root-Locus Design (L7.2)
K_P	0.104	0.10354
K_I	2.07	2.0708

Table 1: Comparison of base and root-locus PI gains for the T1a target system.

The designed gains were essentially identical to the base gains, with only minor differences due to rounding. Therefore, the root-locus design confirmed the controller parameters used for the remainder of the laboratory exercise. A test run of the controller with these gains produced stable closed-loop motor operation.

Steady-state operation in the lab setup was identified using the following observations:

- The motor starts and reaches the commanded speed.
- V_J settles near the reference value (approximately 99–100 rpm).
- V_J does not continually grow, continually decrease, or oscillate wildly.
- VDAout settles to a finite value (approximately 150–250 mV) instead of running to the output limits.
- The motor sound remains smooth, rather than showing violent oscillation or continuous hunting.

The steady-state evidence captured during this test run was shown in Figure 3.

3.3 Problem L7.3

Write, test, and debug the program described in Section L7.2. For debugging purposes, use the base set of parameters.

The PI motor velocity controller described in Section L7.2 was implemented in C and tested on the myRIO platform using the base controller gains $K_P = 0.104$ V s/rad and $K_I = 2.07$ V/rad. Several debugging steps were required to verify proper operation of the velocity measurement, controller calculations, and actuator output. After correcting implementation issues, the controller successfully regulated motor speed. During testing, the measured velocity converged to approximately the commanded speed and the control output remained bounded. As shown in Figure 3 (Figure 3), the final implementation exhibited stable closed-loop operation and was used for the remaining laboratory exercises.

Figure 3: Steady-state controller evidence for the T1a PI velocity-control setup.

A video of this steady-state experiment could be found at https://github.com/juliansoh/ME477/blob/main/Reports/Lab7/videos/Steady-state.mp4.

The program implementation is available at https://github.com/juliansoh/ME477/blob/main/workspace/Julian_lab7/main.c.

3.4 Problem L7.4

Measure the steady-state speed of the motor with an optical tachometer. Compare this value to the reference speed specified in the table. How close is it?

For this test, the reference speed was set to $V_r = 500$ rpm and the system was allowed to reach steady-state operation before measurement. The program display at this operating point was shown in Figure 4. The optical tachometer readings were between 494 and 498 rpm, as shown in Figure 5. This measured range was very close to the commanded value of 500 rpm.

3.5 Problem L7.5

While the motor is at steady-state speed, gently apply a steady manual load torque to the motor shaft. What responses do you observe on the display for measured velocity V_J and control voltage VDAout? Explain.

While a steady manual load torque was applied to the motor shaft, the measured velocity V_J decreased temporarily. The PI controller responded by increasing the control output voltage VDAout. After a short transient, the measured velocity returned close to the reference value while the control voltage remained at a higher level than before the load

Figure 4: Program display at the 500 rpm steady-state test condition.

Figure 5: Optical tachometer reading during the 500 rpm steady-state test.

was applied. This behavior demonstrates the disturbance-rejection capability of the PI controller. The proportional term reacts immediately to speed error, while the integral term removes the steady-state error caused by the applied load torque.

A video of this experiment could be found at https://github.com/juliansoh/ME477/blob/main/Reports/Lab7/videos/L7-5.mp4.

3.6 Problem L7.6

Beginning with the base set of parameters, explore the effect of varying the proportional gain K_P on the transient response. Try smaller and larger values of K_P. What are the effects on oscillation frequency and damping? Explain in terms of the closed-loop transfer-function parameters described in Section 7.5.2.

The proportional gain was varied while the integral gain was held constant at $K_I = 2.07$. Decreasing K_P produced a slower response with less damping and a longer settling time. Increasing K_P produced a faster response with increased damping and a shorter settling time. This behavior was consistent with the closed-loop transfer function, where K_P appeared in the coefficient of the s term and therefore directly influenced the damping ratio of the system.

3.7 Problem L7.7

Beginning with the base set of parameters, explore the effect of varying the integral gain K_I on the transient responses. Try smaller and larger values of K_I. What are the effects on oscillation frequency and damping? Explain in terms of the closed-loop transfer-function parameters described in Section 7.5.2.

The integral gain K_I was varied while the proportional gain was held constant at $K_P = 0.104$. Decreasing K_I resulted in a slower response with lower oscillation frequency and slower error correction. Increasing K_I produced a faster response and a higher oscillation frequency, but it also reduced damping and increased overshoot. This behavior agreed with the closed-loop transfer function because K_I directly affected the natural frequency,

$$\omega_n^2 = \frac{K_a K_M K_I}{J}. \tag{16}$$

As K_I increased, the natural frequency increased, causing the system to respond more quickly but with a greater tendency to oscillate. A summary of these observations was provided in Table 2.

K_I	Response Speed	Oscillation Frequency	Damping
1.0	Slower	Lower	Higher
2.07	Baseline	Baseline	Baseline
4.0	Faster	Higher	Lower

Table 2: Effect of varying K_I on transient-response characteristics.

3.8 Problem L7.8

Using the base set of parameters, record the control output voltage u_a and measured velocity responses for a step change in reference velocity from -200 RPM to $+200$ RPM. In MATLAB, compare these experimental responses with the analytical responses for the continuous-system approximation (see the Lab 7 Appendix summary of the closed-loop model).

The theoretical responses can be calculated using the appropriately scaled MATLAB `step()` command. Plot the analysis over the experimental responses, and use `subplot()` to place both the control-value and measured-velocity plots on the same page. What do you conclude?

For Problem L7.8, we performed the comparison in Python (instead of MATLAB) using the captured dataset in `data/Lab7.csv`. The measured responses (u_a and Ω_J) were

overlaid with the analytical continuous-time closed-loop responses on a two-subplot figure (control on top, velocity on bottom), matching the required analysis format. The resulting comparison plot was shown in Figure 6.

The Jupyter notebook used for this analysis was available at `https://github.com/juliansoh/ME477/blob/main/Reports/Lab7/data/L7-8.ipynb`, and the CSV capture file was available at `https://github.com/juliansoh/ME477/blob/main/Reports/Lab7/data/Lab7.csv`.

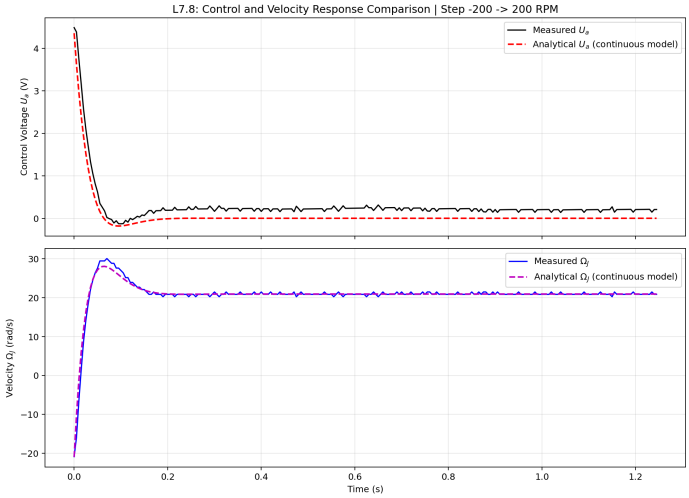

Figure 6: L7.8 comparison of measured responses from `Lab7.csv` and analytical continuous-time model responses generated in Python.

The recorded reference step in `Lab7.csv` was $-200\,\text{RPM} \rightarrow +200\,\text{RPM}$ with $K_P = 0.104$, $K_I = 2.07$, and BTI $= 5\,\text{ms}$. Quantitatively, the comparison gave a velocity RMSE of $0.7748\,\text{rad/s}$ and a control-voltage RMSE of $0.2351\,\text{V}$.

From the overlaid plots, the measured and analytical velocity responses agreed well overall, and the control output showed the same transient trend with small offsets and ripple. The remaining differences were consistent with practical nonidealities such as sampling effects, sensor noise, and hardware limitations.

4 Author Contributions

There was only one author for this report and lab.

References

Rico A. R. Picone, Joseph L. Garbini, and Cameron N. Devine. *An introduction to real-time computing for mechanical engineers*. The MIT Press, 2024.